

SYSTEMCRAFT LANDING PAGE — CREATIVE DIRECTION

You are the lead Product Designer, Motion Designer, Creative Director, and Senior Frontend Engineer responsible for designing the landing page for SystemCraft.

Do not create another generic SaaS landing page.

This website should feel like an interactive product experience rather than a marketing page.

About the Product

SystemCraft is an AI-powered platform where software engineers practice System Design Interviews.

Users don't simply answer questions.

They design distributed systems.

They simulate failures.

They analyze bottlenecks.

They optimize latency.

They receive AI feedback exactly like a real FAANG interview.

The website itself should communicate this philosophy before the visitor even reads the text.

The visitor should immediately feel like:

"I want to build systems."

Design Philosophy

Take inspiration from:

• Apple (clarity, typography, spacing) • Linear (minimalism) • Vercel (motion and gradients) • Stripe (storytelling) • Figma (playfulness) • Arc Browser (premium interactions)

Do NOT copy them.

Create something original.

Avoid generic Tailwind SaaS layouts.

Every section should have its own identity.

Brand Personality

SystemCraft should feel

• Intelligent • Technical • Premium • Futuristic • Calm • Precise • Interactive

Never feel flashy.

Never feel noisy.

Never feel corporate.

Never feel like a template.

Visual Language

Dark mode only.

Deep space inspired.

Infinite grid.

Subtle stars.

Noise texture.

Volumetric lighting.

Soft glows.

Gradient lighting.

Glass only where meaningful.

Avoid excessive glassmorphism.

Lots of breathing room.

Large typography.

Strong visual hierarchy.

Generous whitespace.

Color Palette

Primary: Electric Indigo

Secondary: Royal Purple

Accent: Cyan

Background: Near Black

Highlights: Soft White

Status Colors

Success Emerald

Warning Amber

Failure Crimson

Animations should use lighting rather than saturation.

Motion Language

Motion is storytelling.

Nothing should animate simply because it can.

Every animation must communicate state change.

Examples:

Architecture assembling

Traffic flowing

Requests moving

Cache hits

Database overload

Autoscaling

Health recovery

Latency reduction

Packet routing

Load balancing

Node failures

Everything should feel alive.

Hero Section

Do NOT create another hero with text on the left and image on the right.

Instead:

The hero should be a living distributed system.

When the page loads

Servers appear.

Connections draw themselves.

Traffic begins flowing.

Requests travel across nodes.

A cache is introduced.

Latency drops.

One database fails.

Traffic reroutes.

An autoscaler launches.

Everything stabilizes.

Only then should the headline become visible.

The hero itself demonstrates why SystemCraft exists.

Scroll Storytelling

The website should tell a narrative.

Section 1

"What is System Design?"

↓

Section 2

"How distributed systems behave"

↓

Section 3

"Failures happen"

↓

Section 4

"AI Interview begins"

↓

Section 5

"You receive feedback"

↓

Section 6

"Become interview ready"

Each section should transition naturally into the next.

Avoid hard cuts.

Feature Sections

Never use boring feature cards.

Instead build interactive demonstrations.

Examples

Hover a cache

Traffic reroutes instantly.

Hover the database

Latency spikes.

Hover Kafka

Queue grows.

Hover Load Balancer

Traffic redistributes.

Hover CDN

Requests terminate closer to users.

Users should understand concepts without reading paragraphs.

Typography

Typography should carry the design.

Very large headings.

Short paragraphs.

Never exceed 3–4 lines.

Clear visual rhythm.

Generous spacing.

Elegant line heights.

Buttons

Buttons should feel tactile.

Soft magnetic hover.

Subtle lighting.

Depth.

Glow.

No oversized shadows.

No exaggerated scaling.

No bouncing.

Cursor Interactions

Cursor should subtly influence the page.

Background gradients shift.

Particles react.

Cards tilt slightly.

Nodes brighten.

Connection lines glow.

Keep everything restrained.

Scroll Animations

Use scroll to tell a story.

Pin sections where appropriate.

Zoom into architecture.

Reveal infrastructure gradually.

Animate SVG paths.

Animate request packets.

Reveal metrics.

Never animate entire sections simultaneously.

Motion should guide attention.

Microinteractions

Every interaction should have feedback.

Hover

Focus

Click

Loading

Success

Failure

Transitions should feel physical.

Aim for delight without distraction.

Components

Components should be reusable.

Cards

Metrics

Network nodes

Architecture blocks

Animated connectors

Status indicators

Timeline

Dashboard widgets

Pricing cards

Everything should follow one cohesive design language.

Performance

Maintain 60 FPS.

Avoid unnecessary JavaScript.

Respect prefers-reduced-motion.

Use GPU-friendly transforms.

Lazy load expensive effects.

Optimize all animations.

Technology Stack

Next.js App Router

TypeScript

Tailwind CSS

Framer Motion

GSAP ScrollTrigger (for narrative sections)

Lenis (smooth scrolling)

SVG path animations

Canvas/WebGL only where it adds meaningful value

Final Goal

The visitor should not think:

"This is a nice landing page."

They should think:

"I've never seen a website explain System Design like this before."

Every design decision, interaction, animation, transition, layout, and microinteraction should reinforce that feeling.

Create production-quality code.

Avoid placeholders.

Avoid generic UI.

Think like a team shipping a product that could win Website of the Day.